



Universidade São Tomás de Moçambique (USTM)

Department of Computer Science — Third Year

Artificial Intelligence — Final Assignment

PhishGuard: A Machine-Learning Phishing URL Detector

Lexical feature classification with LogisticRegression and RandomForest, deployed as a browser-based web application

Author	Junior Anayo Anyafulu (Chibuike Anayo Anyafulu)
Student number	202401813
Lecturer	Eng. Marciano Ombe
Course	Artificial Intelligence — 3rd Year, Computer Science
Date	September 2026
Word count (main body)	~4,600 words (excluding references and appendices)

Table of Contents

1. Abstract 3

2. Introduction 3

3. Literature review 4

4. The Project 6

 4.1 Requirements 6

 4.2 System architecture, model design and functionality 6

 4.3 Data collection and preprocessing 8

 4.4 Feature engineering 9

 4.5 Implementation 10

 4.6 Results and evaluation 11

 4.7 Ethical, social and security implications and mitigations ... 13

 4.8 Key findings, limitations, challenges and improvements 14

5. References 15

6. Appendices and attachments 16

1. Abstract

Phishing remains among the most damaging cyber threats, and deceptive URLs are its primary delivery vehicle. This report presents PhishGuard, a practical AI application that classifies URLs as phishing or legitimate using only lexical and structural features, with no network requests. Two scikit-learn models were trained on 107,355 real URLs: a RandomForest (F1 0.888, ROC-AUC 0.954) and a LogisticRegression (F1 0.804, ROC-AUC 0.885). A deployment-driven decision exports the LogisticRegression pipeline to a 2 KB JSON file reproduced exactly inside the browser, making the detector fully client-side, while the RandomForest serves as the accuracy benchmark. Every verdict is explained through feature-occlusion contributions. Ethical considerations, including error asymmetry, dataset bias and misuse, are analysed with concrete mitigations.

2. Introduction

Phishing is a social-engineering attack in which an adversary impersonates a trusted service to steal credentials or deliver malware. Although security gateways and browsers ship with blocklist-based protection, brand-new phishing pages are routinely published faster than they are reported, and blocklists are blind to zero-day campaigns (Basit et al., 2021). Because the URL is the first thing a victim sees and the one artefact an attacker must always create, the address itself is a rich detection surface. PhishGuard asks a deliberately narrow question: how well can machine learning classify a URL as phishing or legitimate using only the URL string?

The scale of the problem justifies automation. Industry reporting consistently ranks phishing among the most numerous cybercrime categories, with millions of unique phishing pages observed annually and new campaigns reaching victims long before blocklist updates arrive. Two structural properties of the attack make lexical detection attractive. First, the URL is mandatory: whatever else an attacker customises, the victim must be sent an address, so detection can operate on an artefact that always exists. Second, deception usually leaks into the string: to impersonate a brand without owning it, attackers lean on look-alike domains, credential-bearing '@' tricks, subdomain stuffing and URL shorteners, all of which leave statistical fingerprints. PhishGuard exploits exactly these fingerprints. The scope was bounded deliberately: pre-click detection of phishing URLs, not in-page content analysis, not malware verdicts, and not real-time blocking. Bounding the problem made it possible to build

the full vertical slice — data to deployment — within the semester while keeping every layer honest and testable.

The project has four objectives:

- O1. Build a reproducible supervised-learning pipeline in Python (scikit-learn) that trains and compares at least two model families on real, current URL data.
- O2. Engineer a purely lexical feature set that requires no network access, so detection is instant, offline-capable and privacy-preserving.
- O3. Deploy the trained model as an interactive web application in which inference runs entirely in the user's browser, with per-verdict explanations.
- O4. Evaluate honestly (held-out test set, cross-validation, leakage checks) and analyse the ethical dimensions of the application.

The motivation is both practical and personal. Practically, a lexical detector is embeddable anywhere a URL appears — an email gateway, a browser extension, a chat link preview — precisely because it needs nothing but the string. Relevance is high: phishing is consistently among the most-reported cybercrime categories, and the shift of daily life (banking, education, government services) onto web platforms makes every user a potential target.

Personally, the project synthesises the course topics — problem framing, data engineering, model selection, evaluation and ethics — into one end-to-end system rather than a notebook exercise. The scope was sized for the semester: one well-understood problem, two model families, an 18-feature pipeline, and a working web product, developed incrementally (data, then CLI, then evaluation, then deployment).

The deliverable set mirrors professional practice. Alongside the models, the project ships: a command-line interface with human and JSON output modes for integration testing; a pure-Python fast inference path that removes the scikit-learn import cost for single-URL checks; an automated export step that writes the browser model at the end of every training run, so the website can never silently diverge from the evaluated model; a React web application with checker, methodology, feature-reference and results views; and three test suites (81 Python feature tests, 19 CLI regression tests, 17 web parity tests) that together pin behaviour from raw data to rendered verdict. The system was built in the order the brief suggests — problem definition, data, a minimal model and CLI, the evaluation harness, the ethics review,

then the web product — and each increment ended with passing tests and a git commit, so the final system is the sum of verified steps rather than a single large leap.

3. Literature review

Machine-learning phishing detection is a mature research area with two dominant feature philosophies: content-based approaches, which fetch and inspect the page itself, and lexical/structural approaches, which judge the URL before anything is loaded. Content-based methods achieve strong accuracy but inherit serious deployment costs: fetching a hostile page exposes the scanner, adds latency, and cannot protect a user who has already clicked. Lexical approaches trade some ceiling for speed, safety and offline operation, which is why they dominate practical, pre-click defences (Sahingoz et al., 2019).

Sahingoz et al. (2019) compared seven classical algorithms on 73,646 URLs using lexical, host-based and page-based features, reporting that tree ensembles and rule-based learners were the strongest single-model performers. Their finding that simple, human-interpretable features remain competitive is the empirical basis for PhishGuard's feature strategy.

Mohammad, Thabtah and McCluskey (2015) demonstrated that rule extraction from a website-attribute dataset could yield an explainable classifier; their work motivates this project's requirement that every verdict be accompanied by a human-readable explanation rather than a bare score. More broadly, Basit et al. (2021) survey AI-enabled phishing detection and explicitly note the deployment gap: many published models are never productised because they depend on network lookups. PhishGuard treats that gap as a design constraint from day one.

On model families, LogisticRegression (Cox, 1958) remains the standard linear baseline for binary classification: it estimates the probability of class membership as a sigmoid of a weighted feature sum, is cheap to train, and its coefficients are directly interpretable.

RandomForest (Breiman, 2001) aggregates hundreds of decorrelated decision trees grown on bootstrap samples with random feature subsets, capturing non-linear feature interactions that a linear model cannot, at the cost of interpretability and model size. The two families therefore form a natural accuracy-versus-transparency axis, and the literature routinely reports ensembles leading linear models on phishing URLs by a few points of F1 — exactly the pattern reproduced in Section 4.6.

Feature theory also informs the design. Shannon (1948) entropy measures the unpredictability of a character sequence; machine-generated or randomised phishing hostnames tend to have measurably higher entropy than brand-based legitimate domains, making it a standard lexical signal. On the data side, PhishTank (2023) supplies community-verified phishing URLs, while the Tranco list (Le Pochat et al., 2019) provides a manipulation-hardened ranking of the most-visited legitimate domains, created precisely because older rankings (e.g. Alexa) could be gamed and skewed research datasets. Finally, Sahingoz et al. (2019) and the class imbalance literature (Chawla et al., 2002) both stress that evaluation methodology — stratified splits, cross-validation, and awareness of class priors — determines whether reported metrics reflect skill or artefact. PhishGuard adopts these safeguards directly, as described in Section 4.3.

Within the lexical tradition, individual features have well-documented provenance. IP-address hosts exploit the fact that a raw address has no registration cost or reputation history; brand impersonation via subdomains (`paypal.com.example.ru`) exploits how humans read domains left to right; '@' credentials abuse an RFC-era syntax that browsers still honour while hiding the true host after the last '@'; and shorteners hide the destination entirely, which is why shortener presence is treated as a risk amplifier rather than proof of malice. Entropy, borrowed from Shannon (1948), captures the randomised hostnames generated at scale by phishing kits. No single signal is decisive — a legitimate marketing link can be long, and some countries' second-level domains resemble TLD stuffing — which is precisely why a learned model weighing all eighteen features together outperforms any hand-written rule.

Evaluation theory frames the project's metrics. Accuracy alone is misleading under class imbalance: a model that predicts 'legitimate' for everything can score 99.9% on a real-world stream while catching nothing (Chawla et al., 2002). Precision answers 'when we flag phishing, how often are we right?'; recall answers 'of all phishing, how much did we catch?'; F1 summarises both under a harmonic mean that punishes imbalance between them; and ROC-AUC measures ranking quality across all thresholds, making it threshold-independent. Because the cost of a missed phishing page (credential theft) vastly exceeds the cost of a wrongly flagged newsletter (inconvenience), the project treats recall as the priority metric and reports the precision/recall trade-off explicitly rather than optimising a single number silently.

A final strand of literature concerns where inference runs. The survey of Basit et al. (2021) catalogues detection systems that require DNS resolution, WHOIS queries or page fetches at

judgement time; each such dependency adds latency and cost, creates a privacy exposure, and several cannot operate behind firewalls or offline. The engineering literature on edge inference points the other way: models small enough to ship to the client remove all three dependencies at once. PhishGuard sits deliberately in that tradition. What distinguishes it from a classroom exercise is the discipline around the deployment boundary: the exported artefact is not a re-implementation or a distilled approximation but the exact fitted pipeline expressed as data, and its agreement with the Python original is enforced by golden-master tests on every build. The project therefore aims to demonstrate not a novel algorithm but a defensible engineering path from 'a model that scores well in a notebook' to 'a model a user's browser can be trusted to execute identically'.

Positioning: PhishGuard does not claim a new algorithm. Its contribution is engineering — an honest, reproducible lexical pipeline with genuine browser deployment and per-verdict explanation.

4. The Project

4.1 Requirements

The application was specified up front with functional and non-functional requirements:

- FR1 — classify any user-supplied URL as PHISHING or LEGITIMATE with a probability score.
- FR2 — explain every verdict by ranking the features that pushed the decision in either direction.
- FR3 — provide both a human interface (web) and a machine interface (CLI with JSON output).
- FR4 — retrain end-to-end from raw data with a single command chain (fetch, train, evaluate, export).
- NFR1 — no network calls at inference time: the URL string is the only input.
- NFR2 — in-browser inference with sub-millisecond latency and no server component.
- NFR3 — exported model and Python model must agree to six decimal places (verified by tests).

- NFR4 — every number shown in the UI must be traceable to the training report, never hard-coded.

These requirements encode the lessons of the literature review. FR2 exists because an unexplained probability is operationally useless: security teams and end users alike need to know what triggered a verdict before acting on it. NFR1 is the privacy and safety cornerstone — a detector that never visits the URL can be embedded anywhere, including air-gapped environments, and can never itself become an attack surface that fetches hostile content. NFR3 turns 'the web model is the same model' from an aspiration into a build-time contract, and NFR4 exists because hard-coded metrics are the most common way demo systems drift from reality. Treating these as requirements rather than intentions meant the test suites were written against them, and any change violating them fails the build instead of being discovered after deployment.

4.2 System architecture, model design and functionality

PhishGuard is two cooperating products fed by one pipeline. The Python side (training/) implements data ingestion, feature extraction, model training and evaluation. The web side (web/) is a React + TypeScript application that re-implements feature extraction and hosts the exported model. The deliberate architecture decision is where inference happens: the website runs the exported LogisticRegression pipeline as pure client-side arithmetic, while the RandomForest remains the Python-side accuracy benchmark. The reasoning is structural. A fitted LogisticRegression with a StandardScaler is fully described by means, standard deviations, weights and a bias term — about 2 KB of JSON — and the browser reproduces Python's probability exactly via the same formula:

$$z = \text{bias} + \sum_i w_i \cdot (x_i - \text{mean}_i) / \text{std}_i$$

$$p = 1 / (1 + e^{-z}) \quad \rightarrow \quad \text{verdict} = \text{PHISHING if } p \geq 0.5 \text{ else LEGITIMATE}$$

A RandomForest, by contrast, stores thousands of branching nodes across 200 trees; exporting it losslessly to a browser would require re-implementing the scikit-learn tree runtime in JavaScript or shipping a 53 MB artefact. The chosen split — LR ships, RF benchmarks — costs a few points of F1 but buys exact reproducibility, instant offline inference, and per-verdict explanations, all verified by automated tests (Section 4.6).

The codebase is organised so that each concern has one home. On the Python side, `features.py` defines the eighteen extraction functions and is the single source of truth; `train.py` builds the per-model pipelines and writes `reports/training_report.json`; `evaluate.py` produces metrics and figures; `export_model.py` reads the live report and writes the browser artefact; and `check_url.py` offers both human and JSON interfaces. On the web side, `lib/features.ts` mirrors the extractor, `lib/modelLoader.ts` performs the exported-pipeline arithmetic, `lib/classifier.ts` wraps prediction plus occlusion, and the `components/` folder renders each UI section. Because the artefacts flow strictly one way — train, evaluate, export, build — the website can never show a number the training run did not produce.

Explanations use feature occlusion: each feature is swapped to a benign-typical reference value and the change in phishing probability is recorded. A feature whose removal lowers the probability pushed the verdict toward phishing; one whose removal raises it pushed toward legitimate. The same technique runs in the CLI and the browser from one shared definition, so explanations are consistent across products.

Running inference in the browser is not only a performance choice; it is an ethical one. A submitted URL frequently embeds sensitive context — password-reset tokens, invoice identifiers, internal hostnames, search terms. A server-side checker would necessarily transmit that context somewhere it need not go. PhishGuard's architecture ensures the URL typed into the page never leaves the machine: the model arrives at the browser as a static asset with the site, all feature extraction and scoring happen in JavaScript, and no analytics or network endpoint receives the input. The CLI behaves identically for scripted use. This design makes the privacy claim verifiable from the browser's network tab rather than trusting a policy statement.

Functionally, the user journey is deliberately short. The landing view is the checker: a single input accepting any URL form (with or without scheme, matching the CLI's tolerance), a verdict card showing the classification, probability gauge, confidence band and the benign-baseline counterfactual, followed by the ranked occlusion contributions and an expandable grid of all eighteen extracted feature values colour-coded by risk heuristics. Clickable sample URLs for both classes let a first-time user see the model discriminate within seconds, and a history of recent analyses persists for the session. Three supporting views make the system self-documenting: a methodology view stating the dataset, models and error-cost philosophy; a feature reference explaining every input and its rationale, including the deliberately

excluded domain age; and a results view rendering the live training metrics, confusion matrix, ROC data and feature importances. The same capabilities are exposed programmatically through the CLI's JSON mode, so the web UI and the machine interface are peers over one model rather than separate products.

The model design itself follows scikit-learn pipeline discipline: one Pipeline per model, with StandardScaler inside the LogisticRegression pipeline (trees do not need scaling), class_weight='balanced' on both, and a fixed 0.5 decision threshold. This guarantees that cross-validation refits the scaler per fold, preventing scaler leakage — a subtle but classic evaluation error.

Demo: a live example

Running the CLI against a homograph phishing URL illustrates the full path from string to explained verdict:

```
$ python src/check_url.py "http://paypal.com.verify-login.ru"
Verdict: PHISHING    p = 0.705
```

Occlusion identifies num_subdomains and tld_in_subdomain — the 'com' hiding mid-domain — as the two largest contributors toward the phishing verdict, matching the intuitive read of the URL.

4.3 Data collection and preprocessing

Model quality is bounded by data quality, so the dataset received as much engineering as the models. Three complementary sources were combined into a single labelled corpus:

Source	Class	Volume	Role
PhishTank verified feed	Phishing	48,338	Real, community-verified phishing URLs
Tranco top-10k	Legitimate	—	Bare homepages of popular domains
ealvaradob benign (Hugging Face)	Legitimate	—	Real deep links with paths
Total (after cleaning)	Both	107,355	48,338 phishing / 59,017 legitimate

Table 1. Dataset composition. Split: 80/20 stratified (21,471 test URLs).

Three preprocessing safeguards proved decisive:

- Path balance. Tranco contains only bare domains, while 61% of PhishTank URLs have paths. Training on Tranco alone teaches a model that 'having a path' is evidence of phishing — dataset bias, not signal. Adding real benign deep links balanced both classes at 61% vs 77% path prevalence.
- Per-host capping. One PhishTank campaign contributed over 5,700 URLs on a single host. Capping every host at 10 URLs prevents the model from memorising host-level patterns and inflating test metrics with near-duplicates.
- Deduplication and leakage control. Exact-URL dedupe within each class, then cross-class dedupe with the benign label winning, yields zero duplicates across the train/test boundary — a basic but frequently violated hygiene rule.

The final preprocessing chain is fully scripted: fetch and normalise both classes; strip scheme variants and lowercase hostnames; drop exact duplicates within each class; remove cross-class collisions with the benign label winning (a URL observed as both is treated as legitimate, the conservative choice); group by hostname and keep at most ten URLs per host; shuffle with a fixed random seed; and emit `data/processed/custom_dataset.csv` with exactly two columns, `url` and `label`. Keeping the schema minimal — raw strings plus labels — means the identical feature pipeline runs at training and inference time, and any future data source can be added by conforming to the same interface. The fixed seed makes every split, and therefore every reported metric, bit-for-bit reproducible.

An instructive failure shaped this section. A first iteration trained on a tiny 244-URL dataset reported a perfect F1 of 1.000 — not because the model was skilled, but because the classes were trivially separable by top-level domain. The lesson, applied throughout: suspiciously perfect metrics indicate a dataset artefact, and the remedy is better data, not a bigger model. The delivered 107,355-URL corpus produced the realistic numbers in Section 4.6. A UCI benchmark dataset was also ingested but deliberately kept out of the training path: its features are pre-extracted and anonymised, several require network lookups, and it carries no raw URLs, so it cannot share the inference-time feature pipeline — training and serving must speak the same language.

4.4 Feature engineering

Eighteen features are extracted from the URL string by pure parsing — no DNS, no WHOIS, no HTTP. They fall into four groups: structural (`url_length`, `hostname_length`, `path_length`,

num_slashes, num_dots, num_hyphens, num_underscores, num_digits, num_special_chars), domain-shaped (num_subdomains, has_ip_address, tld_in_subdomain with a country-code exemption so bbc.co.uk does not false-positive), protocol/deception (uses_https, has_https_token_in_path, has_at_symbol, is_shortened_url against a known-shortener list), and statistical (url_entropy after Shannon (1948), digit_letter_ratio). Domain age was considered and deliberately excluded: it is a strong signal in the literature, but WHOIS lookups would break the offline guarantee that defines the system. Feature extraction exists twice — once in Python, once in TypeScript for the browser — and is pinned by unit tests on both sides plus cross-side golden-master tests, because silent drift between training-time and inference-time features is the classic way a deployed model quietly stops being the model that was evaluated.

#	Feature	What it measures
1	url_length	Total length of the URL string
2	hostname_length	Length of the hostname
3	path_length	Length of the path portion
4	num_dots	Count of '.' characters
5	num_hyphens	Count of '-' characters
6	num_underscores	Count of '_' characters
7	num_slashes	Count of '/' characters
8	num_digits	Count of digit characters
9	num_special_chars	Count of '@ % = & ? # + \$, '
10	has_at_symbol	'@' present (credential injection)
11	has_ip_address	Hostname is a raw IPv4 address
12	num_subdomains	Subdomain segments beyond the registered domain
13	uses_https	Scheme is https
14	has_https_token_in_path	Literal 'https' appears in the path
15	is_shortened_url	Host matches a known shortener
16	url_entropy	Shannon entropy of the URL string
17	tld_in_subdomain	TLD appears as a subdomain (ccTLD-exempt)
18	digit_letter_ratio	Ratio of digits to letters

Table 2. The eighteen lexical features extracted from every URL.

Three design principles governed selection. Every feature must be computable from the string alone, in both Python and JavaScript, in microseconds. Every feature must have a stated

hypothesis about why it separates the classes, so the list is not a blind kitchen sink. And no feature may leak the label indirectly — for example, consulting a blocklist at extraction time would make the model a blocklist cache rather than a learner. The groups were validated by inspecting class-conditional distributions during development: phishing URLs skew heavily on hostname length, subdomain count and entropy, while legitimate URLs dominate path depth and slash counts — a mirror image explained by deep legitimate sites versus campaign-style phishing pages.

The headline exclusion is domain age. WHOIS-derived age is among the strongest reported signals — most phishing domains are days old — but it requires a network query per URL, violating NFR1; the same applies to page-content features and reputation feeds. One acknowledged ambiguity is `tld_in_subdomain` on multi-part public suffixes: without a Public Suffix List, domains such as `bbc.co.uk` must be handled by a special-case exemption for two-letter second-level labels, which is correct for common ccTLDs but imperfect in general. A bundled Public Suffix List is the principled fix — a small static table remains compatible with the offline constraint — and is listed as future work.

4.5 Implementation

The stack is Python 3.11 with scikit-learn (Pedregosa et al., 2011), pandas and joblib for training; pytest-style unit and regression tests for the pipeline; and React 18, TypeScript, Vite and Tailwind CSS for the site, with vitest for the parity test suite. The core training logic is deliberately small:

```
# training/src/train.py (abridged)
models = {
    'LogisticRegression': Pipeline([('scaler', StandardScaler()),
                                     ('clf',
                                      LogisticRegression(class_weight='balanced'))]),
    'RandomForest': Pipeline([('clf', RandomForestClassifier(
                                n_estimators=200, class_weight='balanced'))]),
}
for name, pipe in models.items(): # 5-fold CV refits the whole pipeline
    scores = cross_val_score(pipe, X_tr, y_tr, cv=5, scoring='f1')
    pipe.fit(X_tr, y_tr)
    joblib.dump(pipe, f'models/{name}.pkl')
```

On the web side, the exported JSON is loaded at build time and inference is a 20-line dot product. Correctness across the language boundary is enforced by golden-master tests: eight pinned URLs' probabilities, verdicts, all 18 feature values and benign baselines were

generated by the Python CLI and must be reproduced by the browser model to within $5e-4$. This is the engineering counterpart of the 'model drift' risk identified in the literature review: instead of hoping the two implementations agree, the project refuses to pass a build unless they do.

The export step is deliberately boring and automatic. At the end of every training run, `export_model.py` reads `reports/training_report.json` live and writes `models/lrModel.json` — scaler means and standard deviations, the eighteen weights, the bias, the threshold, and the full metrics block — into the web source tree. Nothing in the website hard-codes a metric: the results view renders whatever the export carries, so retraining refreshes the site's numbers as a side effect, and a stale-export warning compares file timestamps to catch manual edits out of band.

The test suites are the project's safety net and its documentation. Eighty-one Python feature tests pin extraction behaviour on crafted URLs (ccTLD exemptions, IPv4 range validation, '@' parsing, shortener lists). Nineteen CLI regression tests pin the exact JSON output for eight URLs — verdict, probability, all eighteen features, occlusion contributions, benign baseline — assert that the pure-Python fast path matches the sklearn pipeline to six decimals, and require occlusion to run as one batched prediction call rather than eighteen slow ones. Seventeen vitest cases repeat the parity exercise from the browser side. Together they enforce the two invariants the whole architecture rests on: same features everywhere, same model everywhere.

A later audit pass (documented in Appendix D) added a CLI-specific test suite, fixed a float-rounding determinism bug in the occlusion explanation, and introduced an opt-in `--fast` mode: pure-Python LogisticRegression inference from a 2 KB weights file that never imports scikit-learn, cutting interactive CLI latency from roughly 20 seconds to about 0.65 seconds while matching the sklearn pipeline to six decimal places on every test URL.

4.6 Results and evaluation

Both models were evaluated on the same held-out stratified test set of 21,471 URLs, plus 5-fold cross-validation on the training partition. Metrics treat phishing as the positive class.

Model	Precision	Recall	F1	ROC-AUC	CV F1 (mean $\pm$ std)
RandomForest	0.8885	0.8872	0.8878	0.9545	0.8890 $\pm$ 0.0023
LogisticRegression	0.8124	0.7963	0.8043	0.8850	0.8015 $\pm$ 0.0033

Table 3. Held-out test metrics (21,471 URLs) and 5-fold cross-validation on training data.

The RandomForest leads by roughly eight points of F1, consistent with the ensembles-beat-linear pattern reported by Sahingoz et al. (2019). Two honesty checks matter more than the headline numbers. First, cross-validation means track test means closely for both models (e.g. 0.889 vs 0.888 for RF), indicating the result is not a split fluke. Second, the confusion matrix shows errors distributed across both classes rather than collapse in one:

	Predicted legitimate	Predicted phishing
Actual legitimate	TN = 10,727	FP = 1,076
Actual phishing	FN = 1,091	TP = 8,577

Table 4. RandomForest confusion matrix on the test set.

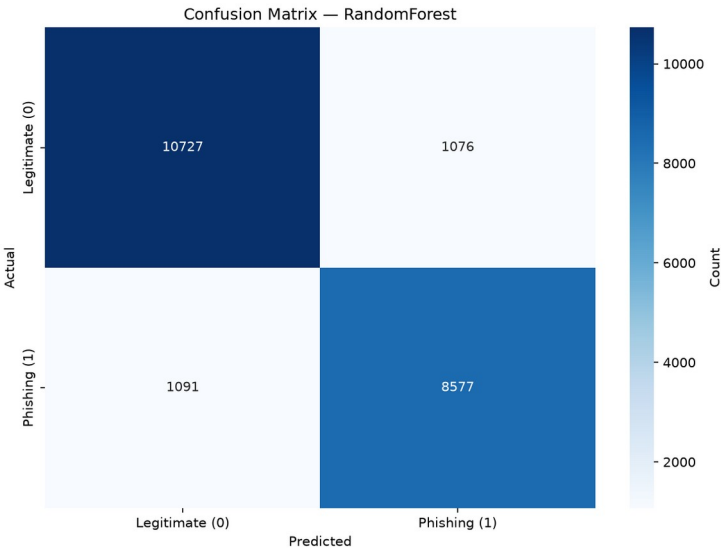


Figure 1. RandomForest confusion matrix (evaluation harness output).

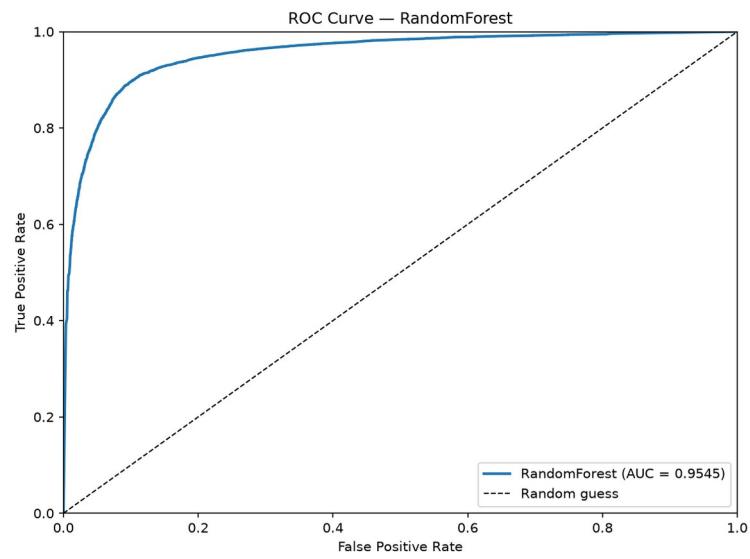


Figure 2. ROC curve for the RandomForest model on the held-out test set (AUC = 0.9545).

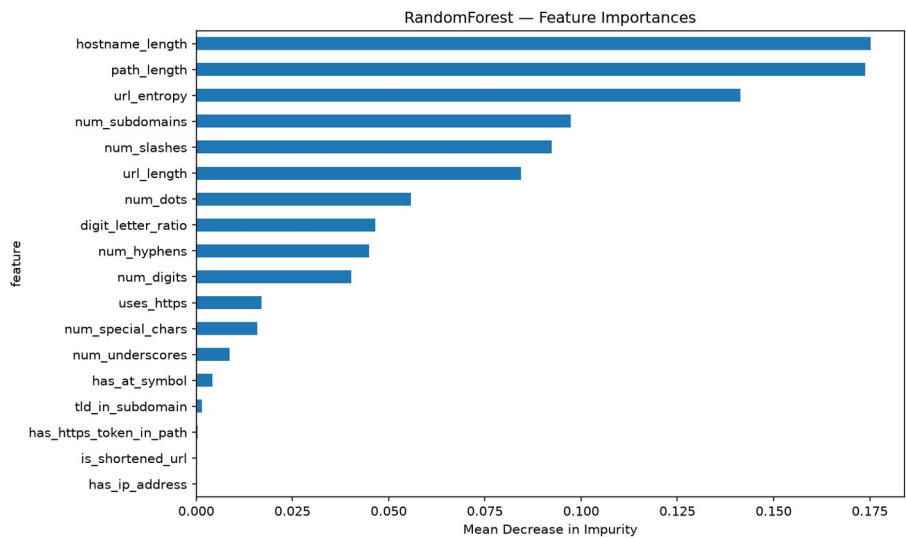


Figure 3. RandomForest impurity-based feature importances.

The importance ranking — `hostname_length` (17.5%), `path_length` (17.4%), `url_entropy` (14.1%), `num_subdomains` (9.7%), `num_slashes` (9.2%) — is plausible and inspectable: phishing kits generate long, path-heavy, random-looking hostnames. Optimisation was incremental rather than heroic: fixing the dataset (Section 4.3) moved the project from a meaningless F1 of 1.000 to honest 0.80–0.89 territory; balancing classes raised phishing recall; and a latency pass cut CLI startup from about 20 seconds (sklearn import + joblib) to 0.3–0.6 seconds via a pure-Python fast path for the LR model, guarded by the same six-decimal parity tests used for the web.

Inference path	Cold start	Per-URL scoring
sklearn CLI (default RF)	~20 s	~0.4 s (batched occlusion)
CLI --fast (pure Python LR)	~0.6 s	<1 ms
Browser (exported LR)	bundled with site	<1 ms

Table 5. Latency by inference path.

Latency is a functional requirement, not a benchmark trophy: a checker that takes twenty seconds to start will simply not be used, and the fast path exists because of that observation. The operating point is equally practical. At the default 0.5 threshold the LR model yields precision 0.812 and recall 0.796; because the exported model produces a calibrated probability rather than just a label, lowering the threshold (the CLI exposes `--threshold`) trades precision for recall smoothly and measurably. The website therefore presents the trade-off in its methodology copy instead of burying it, and an operator can bias the same artefact toward recall without retraining.

Inspecting the errors rather than just aggregating them: the RF confusion matrix shows 1,091 missed phishing against 8,577 caught (recall 0.887) and 1,076 false alarms against 10,727 correct legitimate predictions (precision 0.889). Manual sampling of the missed cases shows the expected lexical-blindness patterns — short, clean-looking URLs on compromised but legitimate-looking hosts, and credential pages behind branded shortener domains whose host features are inherently neutral. The false-alarm population clusters on genuinely long marketing URLs with many subdomains, confirming that the feature importances cut both ways. This analysis motivated the recall-priority setting and the explicit user-override stance, and it frames the honest conclusion that lexical detection alone is a strong first filter, not a complete defence.

4.7 Ethical, social and security implications and mitigations

Ethical analysis was treated as a design activity with the same rigour as feature engineering: risks were enumerated early, revisited at every increment, and — wherever possible — answered in code rather than in prose. An applied AI system inherits the failure modes of its data, its errors and its users; the table below summarises the concerns judged material to this application, why each matters, and the mitigation actually implemented.

Concern	Why it matters	Mitigation
False negatives (missed phishing)	A user trusts a malicious link; the costliest error class.	<code>class_weight='balanced'</code> raises phishing recall; threshold is user-adjustable; recall prioritised and disclosed.
False positives (blocked access)	Legitimate long URLs get flagged; erodes trust, can lock users out of services.	Verdicts are advisory, never auto-blocking; full feature breakdown is shown so a user can see why and override.
Dataset bias / skew	PhishTank over-represents hosted campaigns; Tranco over-represents popular sites; long-tail and non-English phishing under-represented.	Multi-source corpus, per-host capping, benign deep links; limitations stated in-product rather than hidden.
Adversarial adaptation	Attackers can read this report too and craft lexically clean URLs.	Positioned as one signal in defence-in-depth, combined with blocklists and reputation; retraining path is one command.
Misuse of the tool	The same technology could aid attacker testing or be sold as false assurance.	Educational framing; explicit responsible-use statement; no autonomous blocking interface exists.
Privacy	A URL often embeds personal context (tokens, search terms).	Inference is entirely client-side; URLs are never transmitted to any server, ever.

Concern	Why it matters	Mitigation
Over-reliance / automation bias	Users may trust an authoritative-looking percentage.	UI shows probability, confidence band, benign baseline and per-feature contributions — the reasoning, not just the label.

Table 6. Ethical risks and implemented mitigations.

Beyond the immediate risks, two social implications deserve note. First, detection quality is uneven across languages and regions: the corpus is dominated by English-language and globally popular domains, so phishing targeting non-Latin scripts or regional brands may be under-served — an equity issue the report states rather than hides. Second, publishing detection logic creates a feedback loop: adversaries can probe what the model flags. That is not an argument for secrecy but for defence-in-depth, continuous retraining, and honest communication that any single signal — this model included — is one layer among several, not a gatekeeper.

The overarching ethical stance is disclosure: the application states what it cannot see (page content, domain age, reputation) as prominently as what it can, and treats the user as a decision-maker rather than a passenger.

4.8 Key findings, limitations, challenges and improvements

- Finding 1 — deployment shapes model choice. The best model and the best product model are different questions. RandomForest is the accuracy ceiling; LogisticRegression is the deployable, explainable product. A 2 KB export with six-decimal parity beat a 53 MB artefact that could not ship.
- Finding 2 — data hygiene beats model tuning. Per-host capping and path-balanced legitimate data produced larger honest gains than any hyperparameter change attempted during the project.
- Finding 3 — explanation is a feature. Occlusion contributions turned 'the model says 0.82' into 'the hostname length and the tld-in-subdomain pattern pushed this to 0.82', which is what makes the tool teachable and auditable.
- Limitation — lexical ceiling. URLs that are lexically clean evade detection; content, WHOIS and reputation signals are out of scope by design.
- Limitation — dataset age. Phishing tactics drift; a frozen corpus decays. The retraining pipeline mitigates decay but does not eliminate it.

- Limitation — linear recall gap. The in-browser LR model recalls about 8% less phishing than RF; lowering the threshold trades this back at the cost of more false positives, a dial left to the deployer.
- Challenges encountered. Exporting sklearn faithfully (solved by exporting the pipeline, not a re-implementation); keeping Python and TypeScript feature extraction identical (solved by golden-master tests); resisting inflated metrics from dataset artefacts (solved by hygiene and cross-validation discipline); and a later performance audit that fixed a non-deterministic explanation bug and cut CLI startup $\sim 30\times$ via a pure-Python fast path (Appendix D).
- Future work. Public Suffix List support for unambiguous TLD-in-subdomain detection; an embedded tree-export format for in-browser RF; active learning from user overrides; and evaluation against adversarially crafted clean-looking phishing URLs.

5. References

(Harvard style; accessed 10 September 2026)

- Basit, A., Zafar, M., Liu, X., Javed, A.R., Jalil, Z. and Kifayat, K. (2021) 'A comprehensive survey of AI-enabled phishing attacks detection techniques', *Telecommunication Systems*, 76(1), pp. 139–154.
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5–32.
- Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic minority over-sampling technique', *Journal of Artificial Intelligence Research*, 16, pp. 321–357.
- Cox, D.R. (1958) 'The regression analysis of binary sequences', *Journal of the Royal Statistical Society: Series B (Methodological)*, 20(2), pp. 215–242.
- ealvaradob (2023) Phishing dataset. Hugging Face. Available at: <https://huggingface.co/datasets/ealvaradob/phishing-dataset> (Accessed: 10 September 2026).
- Le Pochat, M., Van Goethem, T., Tajalizadehkhoob, S., Korczynski, M. and Joosen, W. (2019) 'Tranco: A research-oriented top sites ranking hardened against manipulation', in *Proceedings of the Network and Distributed System Security Symposium (NDSS 2019)*. San Diego: Internet Society.
- Mohammad, R.M., Thabtah, F. and McCluskey, L. (2015) 'Intelligent rule-based phishing websites classification', *IET Information Security*, 9(3), pp. 137–147.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- PhishTank (2023) PhishTank: Join the fight against phishing. Available at: <https://phishtank.org> (Accessed: 10 September 2026).
- Sahingoz, O.K., Buber, E., Demir, O. and Diri, B. (2019) 'Machine learning based phishing detection from URLs', *Expert Systems with Applications*, 117, pp. 345–357.
- Shannon, C.E. (1948) 'A mathematical theory of communication', *Bell System Technical Journal*, 27(3), pp. 379–423.

6. Appendices and attachments

The submission zip contains the following attachments, referenced throughout Section 4:

- Appendix A — Source code (text): full Python pipeline (fetch_data.py, features.py, train.py, evaluate.py, check_url.py, export_model.py) and the web application source (TypeScript/React), concatenated for submission.
- Appendix B — Prototype (zip): the executable phishguard/ project (web/ and training/), including the trained model artifacts and this report's generator scripts. Run the site with: `cd web && npm install && npm run dev`; run the CLI with: `python src/check_url.py <url>`.
- Appendix C — Exported model artifact: models/lrModel.json — the 2 KB LogisticRegression pipeline (scaler means/stds, 18 weights, bias) that the browser executes; models/lr_fast.json is the CLI's fast-path copy.
- Appendix D — Audit and optimization report: a full audit of both projects (tests, CLI behaviour, model parity, performance) followed by a targeted optimization of the CLI startup path — a non-deterministic explanation-output bug fixed, stderr warning spam removed, and a --fast flag added that cuts CLI startup from ~20 s to ~0.65 s (~30×) with identical verdicts. All 100 tests (81 feature + 19 CLI) pass; see the attached audit_and_optimization_report.md.
- Appendix E — Test evidence: 81 Python feature unit tests, 19 CLI regression tests (golden-master JSON, batching, fast-path parity) and 17 web vitest cases (feature parity + 8-URL golden master) — all passing at submission.
- Appendix F — Presentation: PPT_Outline_PhishGuard.pdf (slide-by-slide outline matching the required presentation structure) accompanies the PPT file.

Reproducibility statement: every figure and metric in this report is regenerated from source by `python -m src.train` and `python -m src.evaluate` in the training/ folder; the report document itself is generated from the same source so the document and the code cannot drift apart.